

Introduction to the system architecture and performance in the STM32H5 MCUs

Introduction

The **STM32H5** series devices are microcontrollers based on the high-performance Arm® Cortex®-M33 32-bit RISC core. The high-performance Arm® Cortex®-M33 32-bit RISC core is an implementation of Armv8-M with TrustZone® security technology that provides security to the system.

Reaching 1.5 DMIPS/MHz and 4.092 CoreMark/MHz, the Cortex®-M33 achieves the perfect combination between real-time determinism, energy efficiency, software productivity, and system security. It enables the STM32H5 series devices to be the right fit for many high-performance applications.

The STM32H5 series devices bring the best performance and a high level of security with:

- A processor that increases in performance in comparison to other Cortex®-M CPUs
- The digital signal processing (DSP) extension
- A floating-point unit (FPU)
- A coprocessor interface to offload compute intensive operations
- The TrustZone® feature to guarantee the maximum of security

The application note presents the global architecture of STM32H5, along with its memory interfaces and features. These memory interfaces and features provide high flexibility and increased code and data sizes, and enable optimal performance. It also introduces the multimaster architecture, which enhances system performance and offloads the CPU.

This application note presents the results of software demonstrations of the architecture performance of STM32H5 series devices in various memory partitioning configurations (different code and data locations), as well as the performance of the architecture when DMAs are enabled.

This document also provides an overview of the STM32H5 performance using the fast Fourier transform (FFT) as an example. Moreover, it gives an overview of three main setups:

- Internal execution of the FFT with different memory allocation zones
- External execution of the FFT using the OCTOSPI transfer
- The multimaster use of the FFT execution and the DMA transfer

1 General information

This document applies to STM32 MCUs based on the Arm® Cortex®-M processor.

Note: Arm is a registered trademark of Arm limited (or its subsidiaries) in the US and/or elsewhere.



2 STM32H5 series system architecture

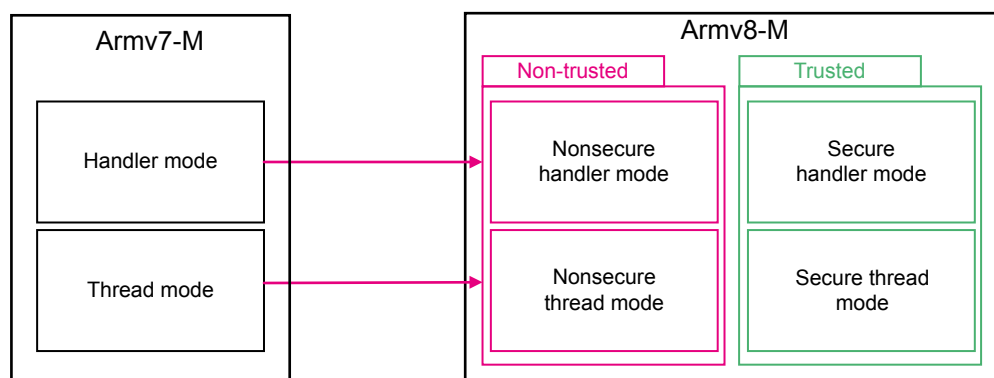
2.1 Cortex®-M33 core

The STM32H5 architecture relies on an Arm® Cortex®-M33 core, which is a highly energy-efficient processor. The Cortex®-M33 operates at up to 250 MHz and features an optional floating-point unit (FPU), which supports single precision arithmetic. It also implements an important set of DSP instructions, a memory protection unit (MPU), and security attribution that enhances the application security. The Cortex®-M33 processor includes a coprocessor interface for frequently used compute-intensive operations. This interface provides a mechanism to increase the general-purpose compute capability. The Cortex®-M33 processor features a 3-stage pipeline. It includes the TrustZone® technology that creates a secure isolated world to provide confidentiality and integrity to the system, protecting it from software attacks.

2.2 TrustZone® security architecture

The Cortex®-M33 processor features the Arm® TrustZone® technology that is based on the Armv8-M security extension. When enabled, it defines the access permissions based on secure and nonsecure states Figure 1.

Figure 1. Armv8-M security extension



DT71600V1

Note: TrustZone® is not available on STM32H503xx devices.

The TZEN option byte in the FLASH_OPTSR2 register activates the TrustZone® security. When the TrustZone® is enabled, the SAU (security-attribution unit) and IDAU (implementation-defined-attribution unit) define the access permissions based on secure and nonsecure states.

When the TrustZone® is enabled, the default security states are the following:

- CPU: The Cortex®-M33 is in secure state after reset. The boot address must be at a secure address.
- Memory map: the SAU is fully secure after reset. As a result, the whole memory map is fully secure. Up to eight SAU configurable regions are available for security attributions.
- Flash memory:
 - Watermark user options define the flash memory security area. All flash memories are fully secure.
 - The flash memory block-based features are nonsecure after reset. Even if the flash memory is configured as nonsecure through IDAU/SAU and through the flash memory secure watermark option bytes, it is possible to configure volatile secure areas using the flash memory block-based feature as secure mode, and this by using the flash memory interface block-based configuration registers.
 - SRAM: the whole SRAM is secure after reset. The block-based memory protection controller (MPCBB) is secure.

The secure memory space is divided into two memory types:

- A secure world, where usually security sensitive applications are run and critical resources are located
- A nonsecure or public world (such as standard nonsecure operating system and user space)

2.3 Instruction and data caches

The devices embed the instruction cache (ICACHE), introduced on the C-AHB bus together with the data cache (DCACHE), introduced on the S-AHB system bus of the Arm® Cortex®-M33 processor.

In most use cases, the ICACHE enables the device to achieve close to zero wait state performance. On another side, the DCACHE enables the device to achieve optimum performance on external memory data accesses.

2.3.1 Instruction cache

The instruction cache (ICACHE) is accessible via two interfaces:

- AHB fast master1 interface with 128-bit data
- AHB slow master2 interface with 32-bit data

The purpose of the instruction cache is to cache instruction fetches or constants data loads coming from the processor. Therefore, ICACHE only manages read transactions and does not manage write transactions.

In order to activate the ICACHE functioning, the EN bit must be set in the ICACHE_CR register.

It caches the code memory region, ranging from address 0x0000 0000 to 0x1FFF FFFF of the memory map.

2.3.2 Data cache

The data cache (DCACHE) is accessible with the AHB master interface with 32-bit data.

The purpose of the data cache is to cache external memories data loads and stores coming from the processor. These accesses include the instruction fetches that may occur to an external memory address. Therefore, DCACHE manages both read and write transactions.

In order to activate the DCACHE functioning, the EN bit must be set in the DCACHE_CR register.

It caches only the external RAM memory region (OCTOSPI and FMC), located from 0x6000 0000 to 0xDFFF FFFF, of the memory map.

Table 1 shows the different cache sizes available on STM32H5.

Table 1. STM32H5 series cache sizes

Device	Instruction cache size	Data cache size
STM32H523xx	8 Kbytes	4 Kbytes
STM32H533xx		
STM32H562xx		
STM32H563xx		
STM32H573xx		
STM32H503xx	8 Kbytes	N/A

2.4 Cortex®-M33 bus interfaces

The Cortex®-M33 has the following interfaces:

- Code AHB (C-AHB) interface
- System AHB (S-AHB) interface

This section describes each of them.

2.4.1 Code AHB (C-AHB) interface

The code AHB (C-AHB) interface is used for any instruction fetch and data access to the code region of the Armv8-M memory map.

The C-bus of the Cortex®-M33 core is connected to the internal flash memory and to the bus matrix via the instruction cache and by means of the fast C-bus. The fast C-bus is used for instruction fetch and data access to the internal memories mapped in the code region. It targets the internal flash memory and the internal SRAMs (SRAM1, SRAM2, and SRAM3). SRAM1, SRAM2, and SRAM3 are accessible on this bus with a continuous mapping.

The C-bus of the Cortex®-M33 core is also connected to the bus matrix via the instruction cache and by means of the slow C-bus. The slow C-bus is used for instruction fetch and data access to the external memories mapped in the code region. It also targets the external memories (FMC and OCTOSPI).

2.4.2 System AHB (S-AHB) interface

The system AHB (S-AHB) interface is used for any instruction fetch and data access to the memory-mapped SRAM, peripheral, external RAM and external device.

The S-AHB bus is connected to the bus matrix via the S-bus. The S-bus is used by the core to access data located in a peripheral or an SRAM area. It targets the internal SRAMs (SRAM1, SRAM2, SRAM3, and BKPSRAM), the AHB1 peripherals including the APB1, APB2, AHB2, AHB3, and AHB4 peripherals. SRAM1, SRAM2, and SRAM3 are accessible on this bus with a continuous mapping.

Note: *The bus matrix has a zero latency when accessing SRAM1, SRAM2, and SRAM3.*

It is also connected to the bus matrix via the data cache by means of the DCACHE S-bus. This bus is used for instruction fetch and data access to the external memories mapped in the data region and targets the external memories (FMC and OCTOSPI).

Note: *Fetching instructions through this bus is less efficient than fetching instructions through the slow C-bus.*

The processor contains a bus matrix that arbitrates instruction fetches and memory accesses from the processor core between the external memory system and the internal system control space (SCS) and debug components. Priority is usually given to the processor to ensure that any debug accesses are as nonintrusive as possible.

The system memory map is Armv8-M Main extension compliant, and is common both to the debugger and processor accesses.

The default memory map provides user and privileged access to all regions except for the private peripheral bus (PPB). The PPB space is privileged access only.

Table 2 shows the default memory map. This is the memory map that is used for implementations without the optional MPUs, or when the included MPUs are disabled. The attributes and permissions of all regions, except that targeting the NVIC and debug components, can be modified using an implemented MPU.

Table 2. Memory access behavior

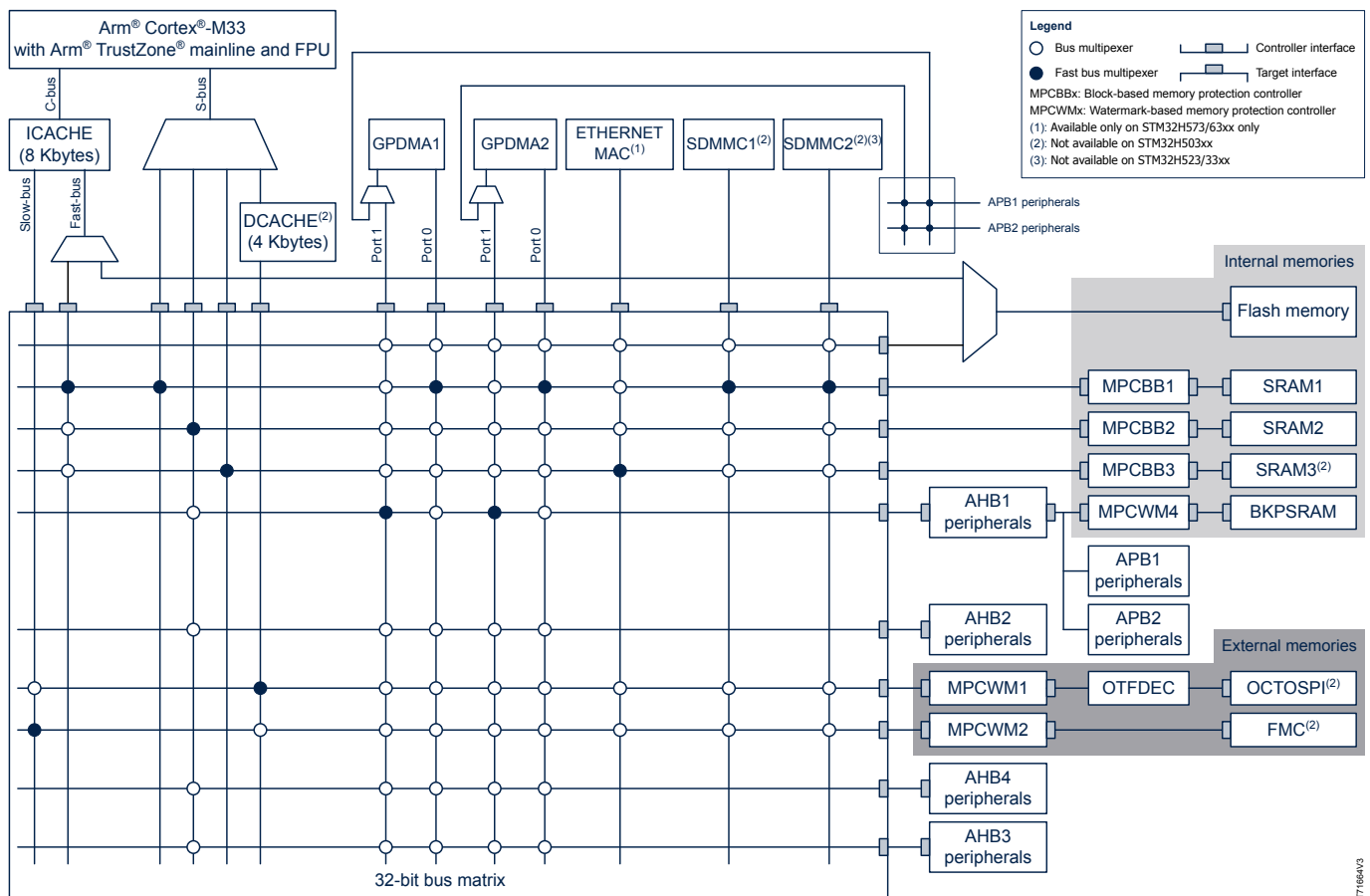
Address range	Memory region	Memory type	Shareability	XN	Description
0x0000 0000-0x1FFF FFFF	Code	Normal	Nonshareable	-	Executable region for program code. Data can also be put here.
0x2000 0000-0x3FFF FFFF	SRAM	Normal	Nonshareable	-	Executable region for data. Data can also be put here.
0x4000 0000-0x5FFF FFFF	Peripheral	Device, nGnRE	Shareable	XN	On-chip device memory.
0x6000 0000-0x9FFF FFFF	RAM	Normal	Nonshareable	-	Executable region for data.
0xA000 0000-0xDFFF FFFF	External device	Device, nGnRE	Shareable	XN	External device memory.
0xE000 0000-0xE003 FFFF	Private peripheral bus	Device, nGnRnE	Shareable	XN	This region includes the SCS, NVIC, MPU, SAU, BPU, ITM, and DWT registers.
0xE004 0000-0xE004 3FFF	Device	Device, nGnRnE	Shareable	XN	This region is for debug components and can include the ETM, CTI, and TPIU configuration registers or none.
0xE004 4000-0xE00F FFFF	Private peripheral bus	Device, nGnRnE	Shareable	XN	This region includes the ROM tables.
0xE010 0000-0xFFFF FFFF	Vendor_SYS	Device, nGnRE	Shareable	XN	Vendor specific.

2.5 STM32H5 bus matrix

The STM32H5 series devices feature a 32-bit multilayer AHB bus matrix that interconnects the core, masters, and slaves. It enables a number of parallel access paths between the core buses, the masters buses, and the slaves buses. Thus, it provides a concurrent access and efficient operation even when several high-speed peripherals work simultaneously. The STM32H5 bus matrix manages the access arbitration between masters. An internal arbiter resolves the conflicts and the bus concurrency of masters on the bus. It uses a round-robin algorithm. This bus matrix features a fast bus multiplexer used to connect each master to a given slave without latency.

Figure 2 shows the overall system architecture of the STM32H5 series devices as well as the bus matrix connections.

Figure 2. STM32H5 series system architecture



The STM32H5 bus matrix interconnects:

- 13 bus masters or initiators
 - Fast C-bus, connecting Cortex-M33 core C-bus to the internal SRAMs through the instruction cache
 - Slow C-bus, connecting Cortex-M33 core C-bus to the external memories through the instruction cache
 - Cortex-M33 core S-bus (three masters connected to three internal SRAMs without latency along with AHB and APB peripherals)
 - Cortex-M33 core S-bus connected to the external memories through the data cache
 - GPDMA1 (general purpose DMA featuring two master reports)
 - GPDMA2 (general purpose DMA featuring two master ports)
 - SDMMC1
 - SDMMC2
 - Ethernet MAC
- 10 bus slaves
 - The embedded flash memory
 - Internal SRAM1 as a default slave
 - Internal SRAM2
 - Internal SRAM3
 - AHB1 peripherals and backup RAM (4-Kbyte BKPSRAM) including AHB to APB bridges and APB peripherals (connected to APB1 and APB2)
 - AHB2 peripherals
 - FMC (flexible memory controller) memory interface
 - OCTOSPI memory interface
 - AHB3 peripherals, including AHB to APB bridge and APB peripherals (connected to APB3)
 - AHB4 peripherals

2.6 STM32H5 memories

The [STM32H5](#) devices embed a flash memory with different sizes depending on the STM32H5 part number. It also embeds SRAMs with different sizes as well as an external memory interface such as the FMC and the OCTOSPI.

This configuration gives the flexibility to users to partition their application memory resources following their needs. It enables them also to get the right compromise performance versus the application code size.

2.6.1 Embedded flash memory

Each device of the [STM32H5](#) series has its own flash memory size (refer to [Section 2.8](#)). The flash memory interface manages the accesses of any master to the embedded nonvolatile memory: It implements the read, program and erase operations, error corrections, as well as various integrity and confidentiality protection mechanisms.

The flash memory interface is accessible via two AHB connections:

- The flash memory AHB register interface
 - It is used to read and write registers (except for some) by 8, 16, and 32 bits. When an unlock sequence for control and option registers is wrong, a bus error is raised, otherwise no read or write errors are generated on the bus.
- The main AHB interface
 - This interface is used to handle three different targets:
 - Code placed in user and system memory, protected by 9 bits of ECC³
 - Secure keys placed in OBKEY sectors, protected by 9 bits of ECC and OTP, read only
 - Flash memory high-cycle data area protected by 6 bits of ECC

By default, the whole AHB memory range is cacheable. For regions where caching is not practical (OTP⁴, RO⁴, data area), MPU has to be used to disable caching.

One of the main features of the flash memory is the prefetch. The prefetch reads the next sequential instruction from the flash memory. The prefetch is enabled by setting the PRFTEN bit in the flash memory access control register (FLASH_ACR). PRFTEN must be set only if at least one wait state is needed to access the flash memory.

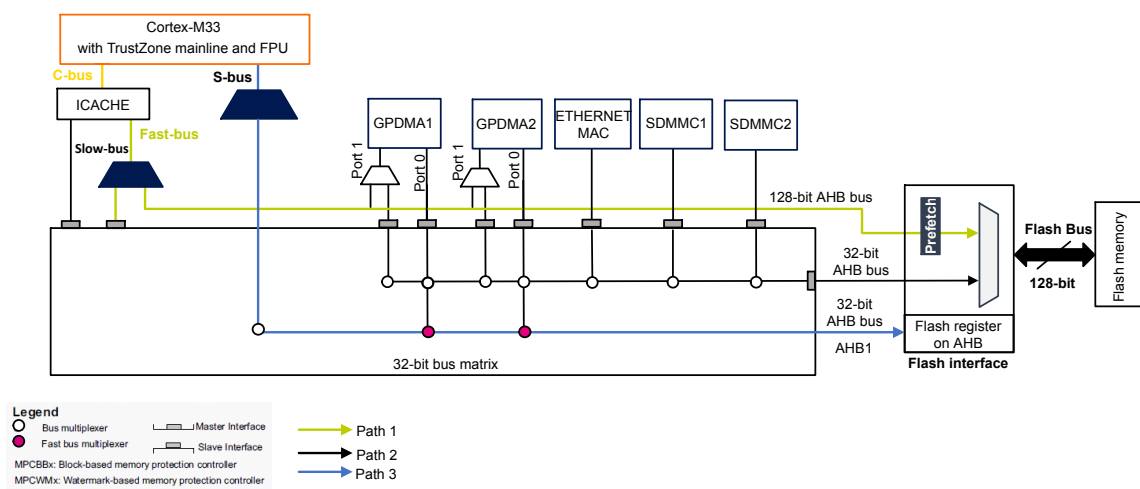
Note: ³Error correction codes (ECC): the embedded flash memory supports an error correction code (ECC) mechanism. It is based on the SECDED algorithm in order to correct single errors and detect double errors.

⁴ OTP (one-time programmable) and RO (read only) memories are accessed through the main AHB interface. The OTP is accessible at the address 0x08FF_F000 to 0x08FF_F7FF and the read only section is accessible from 0x08FF_F800 to 0x08FF_FFFF.

Note: Features availability depends on the part number. Refer to the related datasheet for more information.

Figure 3 shows the possible accesses and connection to the flash memory.

Figure 3. Flash memory interfaces (paths: 1, 2, 3)



DTT1662V1

2.6.2 Embedded SRAM

The STM32H5 series devices feature a large SRAM with a scattered architecture. It is divided into up to four blocks: SRAM1, SRAM2, SRAM3, and BKPSRAM.

These SRAMs are accessible by bytes, half-words (16 bits), or full words (32 bits) and can be addressed both by CPU and DMAs. The CPU can access the SRAM1, SRAM2, and SRAM3 through the S-bus or through the C-bus depending on the selected address. The CPU can access the BKPSRAM through the S-bus only. When the TrustZone® security is enabled, all SRAMs are secure after reset.

Note: All the SRAMs are consecutive in terms of memory location.

SRAM blocks availability depends on the part number. Refer to the related datasheet for more information.

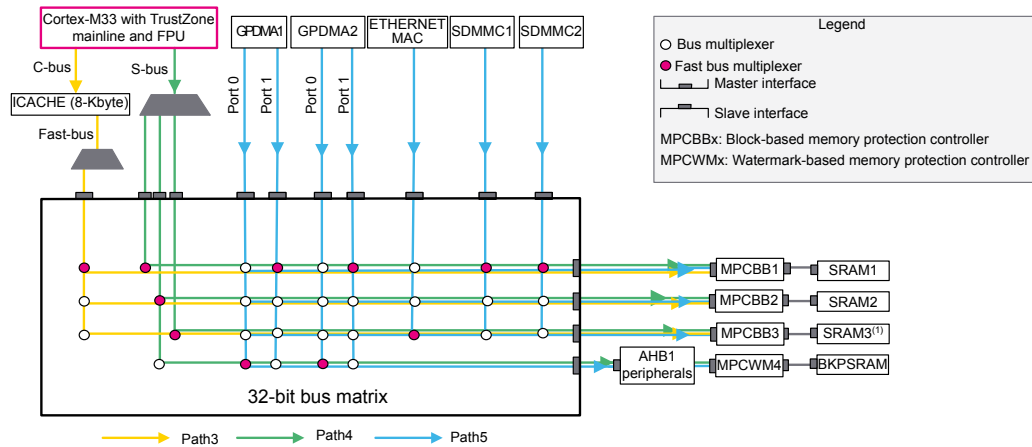
The possible accesses and connection to the different SRAMs are shown in Figure 4.

Path 3 shows the connection between the C-bus of the Cortex-M33 core and the internal SRAMs through the bus matrix.

Path 4 shows the connection of the S-bus of the Cortex-M33 core to the internal SRAMs.

Path 5 shows the connection of the GPDMA, the Ethernet MAC, and SDMMCs buses to the internal SRAMs via the bus matrix.

Figure 4. SRAMs possible connections (paths: 3, 4 and 5)



1. SRAM3 is not available in STM32H503xx devices.

2.6.3 External memories

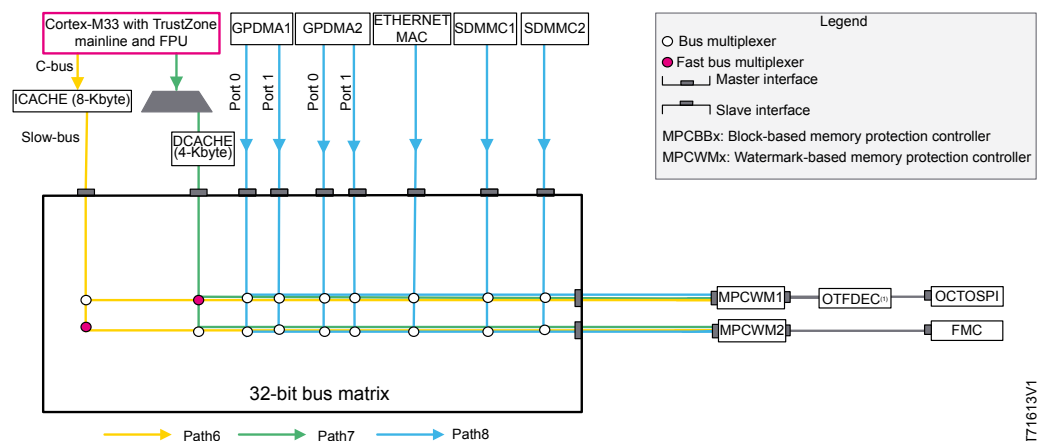
The STM32H5 has several internal memories and storage controllers, such as the USB and the SDMMC. The user can extend these with:

- the flexible memory controller (FMC)
- the OCTOSPI controller

Note that external memories are not available in STM32H503xx devices.

- Figure 5 shows the possible paths that interconnect the CPU and the different DMAs.
- Path 6 shows the connection of the Cortex[®]-M33 core C-bus to the external memories. The connection is made through the instruction cache and through the bus matrix.
- Path 7 shows the connection of the Cortex[®]-M33 core S-bus to the external memories. The connection is made through the data cache through the bus matrix.
- Path 8 shows the connection of the GPDMA, the Ethernet MAC, and SDMMCs buses to the external memories. This connection is made through the bus matrix.

Figure 5. External memory connections

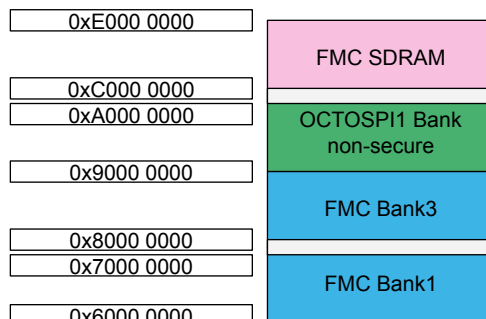


1. The OTFDEC TrustZone-aware peripheral proposes on-the-fly decryption of encrypted images. These are stored on external flash memory that is connected through the OCTOSPI.

All the external memories are accessible by all the masters (CPU and DMAs). Thus, the memory to memory DMAx transfer or peripheral/memory DMAx transfer is allowed.

Figure 6 summarizes the memory mapping of the external memories and their address ranges.

Figure 6. External memory based on IDAU mapping



DT71614V1

2.6.3.1 Flexible memory controller (FMC)

The STM32H5 FMC controller interfaces the memory mapped devices including SRAMs, ROMs, NOR/NAND flash memories and SDRAM devices. Except for NAND flash memory, it is used either for the program execution, or the data load/store operations.

STM32H5 FMC features:

- 4 banks to support different memories at the same time
- Independent chip select control for each memory bank
- Independent configuration for each memory bank
- Programmable timings for each memory bank
- 8-bit, 16-bit data bus width
- External asynchronous wait control
- Interfaces with synchronous DRAM (SDRAM) that provides two SDRAM banks

All the FMC external memories share the addresses, data, and control signals with the controller.

Each external device is accessed by means of a unique chip selected.

The FMC performs only one access at a time to an external device.

All the external memories that are connected to the FMC benefit from the data cache (DCACHE) (path 7 in Figure 5). It allows then to have more data or/and code sizes with the maximum of performance.

2.6.3.2 Octo-SPI interface

The STM32H5 series devices embed an octo-SPI memory interface, which is a specialized communication interface targeting single, dual, quad, or OCTOSPI flash memories. It supports most external serial memories such as serial PSRAMs, serial NAND, and serial NOR flash memories. It supports also hyper RAM and hyper flash memories starting from 0x9000 0000 to 0x9FFF FFFF in the memory mapped mode. Besides, it supports a dual-quad configuration, where eight bits can be sent/received simultaneously by accessing two quad memories in parallel.

In addition, the interface supports single data rate (SDR), which means that the transfer of data is performed only once per clock pulse, and double-transfer rate (DTR), meaning that the data is transferred on both the rising and falling edges of the clock signal. It can work in one of the three following modes:

- Indirect mode: all the operations are performed using the OCTOSPI registers
- Automatic status-polling mode: the external memory status register is periodically read and an interrupt can be generated in case of flag setting.
- Memory-mapped mode: the external memory is memory mapped and it is seen by the system as if it was an internal memory, supporting both read and write operations

Compared to the FMC, the OCTOSPI enables the connection to an external flash memory with a reduced cost, small packages (reducing the PCB area), and a reduced GPIOs usage.

As shown in [Figure 5](#) (path 7), the OCTOSPI is mapped on a dedicated layer on the AHB. It can benefit from the DCACHE. This allows executing the code and loading the data from the OCTOSPI with good performances. This interface is also accessible by all the masters.

The OTFDEC (on-the-fly decryption) adds a layer of security when interfacing external flash memory with OCTOSPI. It is a security feature that uses AES-128 in counter mode to achieve the lowest possible latency, therefore enhancing security of the system without heavily impacting its performance.

When OTFDEC is used in conjunction with OCTOSPI, it is mandatory to access the flash memory using the memory-mapped mode of the flash memory controller.

The OTFDEC allows for up to four independent encrypted regions, with a granularity of the region definition of 4096 bytes. Each region has its own 128-bit key, two bytes firmware version, and eight bytes application-defined nonce, which must be changed each time an encryption is performed by the application.

OTFDEC also supports OCTOSPI pre-fetching mechanism and an enhanced encryption mode to add a proprietary layer of protection on top of AES stream cipher (execute only).

2.7 DMAs

The [STM32H5](#) series devices embed two identical general-purpose direct memory access (GPDMA) controllers that have two ports each. They are used to perform programmable data transfers between memory-mapped peripherals and/or memories. These DMAs are provided to offload the CPU and have eight channels implemented differently. They enable users to allocate and select the best for their application, and this makes the best use of the GPDMA performances.

There are two instances of the GPDMA in the devices, named as GPDMA1 and GPDMA2. Each GPDMA instance has access to the following internal memories: embedded flash memory, SRAM1, SRAM2, SRAM3, and the BKPSRAM.

They also have access to external memories through FMC and OCTOSPI controllers via the bus matrix.

Thanks to the bus matrix architecture, the concurrent access by the CPU and the DMAs to different slaves are managed efficiently in order to minimize the latency.

2.8 Main differences between the STM32H5 series devices

Table 3. Main differences from architecture point of view

Feature	STM32H503xx	STM32H523/533xx	STM32H562/STM32H563/ STM32H573xx
Instruction cache size	8 Kbytes	8 Kbytes	8 Kbytes
Data cache size	Not available	4 Kbytes	4 Kbytes
FPU	VFPv5 single precision floating-point	VFPv5 single precision floating-point	VFPv5 single precision floating-point
Flash memory size	128 Kbytes	512 Kbytes	2 Mbytes
Flash memory width	up to 128-bit access	up to 128-bit access	up to 128-bit access
SRAM1	16 Kbytes	128 Kbytes	256 Kbytes
SRAM2	16 Kbytes	80 Kbytes	64 Kbytes
SRAM3	Not available	64 Kbytes	320 Kbytes
BKPSRAM	2 Kbytes	2 Kbytes	4 Kbytes
Ethernet ⁽¹⁾	Not available	Not available	Available
SDMMCx ⁽²⁾	Not available	Available	Available
FMC ⁽³⁾	Not available	Available	Available
OCTOSPI	Not available	Available	Available

1. Ethernet MAC is available only on STM32H563/573xx devices.

2. SDMMC2 is not available on STM32H523/33xx devices.

3. SDRAM is not available on STM32H523/33xx devices.

3 Typical application

This application note provides two software examples that show, first, the **STM32H5** performance of the CPU memory access either for the data storage or the code execution, and either for internal or external memories. Secondly, the examples show the impact of the DMA usage when the CPU is loading/storing data in a memory in typical scenarios.

The example used is the FFT example provided in the CMSIS library and can be replaced by the user application.

3.1 FFT demonstration

The FFT example is used as it benefits from the floating-point unit of the Cortex-M33. It also contains several loops and data load/store operations, and can be done in different paths/memories.

The code can be executed from internal or external memories. The example consists in the calculation of the maximum energy.

It is calculated in the frequency domain of the input signal with the use of:

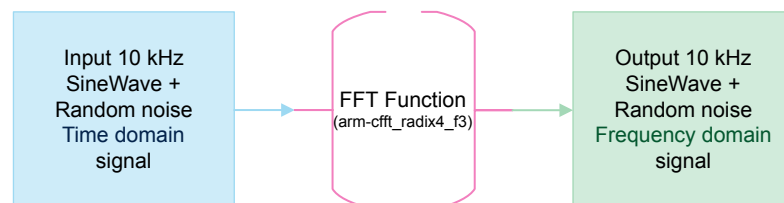
- Complex FFT
- Complex magnitude
- Maximum functions

It uses the FFT 1024 points, and the calculation is based on a single precision floating-point. The input signal is a 10 kHz sine wave merged with white noise.

Figure 7 shows the block diagram of the transformation.

The number of cycles consumed by the FFT process is also calculated based on the SysTick. The results were obtained using EWARM version 6.5 with the C/C++ Compiler 9.50.2.385.

Figure 7. FFT example block diagram



DT71608V1

3.2 Configuration requirements of the CPU access demonstration

To compare various configurations, an understanding of memory organization, the different levels of optimization, and the definition of preprocessor symbols is required. These are detailed in the following sections.

3.2.1 Memory organization

To understand memory organization, consider the following features:

- Memory addresses
- Memory sizes
- Bus connections
- Cache connection and remapping capabilities

Table 4 below shows the memory characteristics for the STM32H5 series.

Table 4. Memory characteristics for the STM32H5 series

Peripheral				Cacheable memory access			Noncacheable memory access		
Type	Name	Product name and size		Bus name	Nonsecure region start address	Secure/ nonsecure callable region start address	Bus name	Nonsecure region starting address	Secure/ nonsecure callable region starting address
Internal	Flash	STM32H503	128 Kbytes	I-cache fast bus	0x0800 0000	N/A		N/A	
		STM32H523/533	512 Kbytes			0x0C00 0000			
		STM32H56x/573	2 Mbytes						
	SRAM1	STM32H503	16 Kbytes		0x0A00 0000	N/A	S-bus	0x2000 0000	N/A
		STM32H523/533	128 Kbytes			0x0E00 0000			0x3000 0000
		STM32H56x/573	256 Kbytes						
	SRAM2	STM32H503	16 Kbytes		0x0A00 4000	N/A	0x2000 4000	0x3002 0000	N/A
		STM32H523/533	80 Kbytes		0x0A02 0000	0x0E02 0000			
		STM32H56x/573	64 Kbytes		0x0A040 000	0x0E04 0000			
	SRAM3	STM32H523/533	64 Kbytes		0x0A03 0000	0x0E03 0000	0x2003 4000	0x3003 4000	
		STM32H56x/573	320 Kbytes		0x0A05 0000	0x0E05 0000			0x2005 0000
	External	FMC SDRAM ⁽¹⁾	STM32H523/533 STM32H56x/573		256 Mbytes	I-cache slow bus	Alias address in the range of [0x0000 0000:0x07FF FFFF] or [0x1000 0000:0x1FFF FFFF]	N/A	S-bus
OCTOSPI 1 bank nonsecure		0x9000 0000							
FMC bank 3 nonsecure		0x8000 0000							
FMC bank 1 nonsecure		0x6000 0000							

1. SDRAM not available on STM32H523/533 devices.

These features are used in the linker file. It maps the memory allocation but also governs every aspect of the linking process, specifying how sections in input files must be mapped in the output file and managing the memory layout.

The table below demonstrates the different linker file extensions.

Table 5. Linker file extensions

EWARM	STM32CubeIDE	Arm® Keil® MDK
.icf (IAR configuration file)	.ld (linker script file)	.sct (scatter file)

The key functions of the linker file are:

- Memory mapping
- Section placement
- Initialization
- Symbol resolution

Figure 8 below shows an example of the linker file (using EWARM) defined for STM32H563/573: the read-only data and code of the program are placed in the flash memory (including the initialization vector), whereas the read-write data are allocated in the SRAM1 (noncacheable addresses).

Figure 8. ICF file mapping memory regions

```

/#####ICF### Section handled by ICF editor, don't touch! ####/
/*-Editor annotation file-*/
/* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_v1_0.xml" */
/*-Specials-*/
define symbol __ICFEDIT_intvec_start__ = 0x08000000;
/*-Memory Regions-*/
define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
define symbol __ICFEDIT_region_ROM_end__ = 0x081FFFFF;
define symbol __ICFEDIT_region_SRAM1_start__ = 0x20000000;
define symbol __ICFEDIT_region_SRAM1_end__ = 0x2003FFFF;

/*-Sizes-*/
define symbol __ICFEDIT_size_cstack__ = 0x7000;
define symbol __ICFEDIT_size_heap__ = 0x18000;
/**** End of ICF editor section. #####ICF###*/

define memory mem with size = 4G;
define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
define region SRAM1_region = mem:[from __ICFEDIT_region_SRAM1_start__ to __ICFEDIT_region_SRAM1_end__];

define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };

initialize by copy { readwrite };
do not initialize { section .noinit };

place at address mem:__ICFEDIT_intvec_start__ { readonly section .intvec };

place in ROM_region { readonly };
place in SRAM1_region { readwrite,
                      block CSTACK, block HEAP };

```

DT75937V1

By modifying the linkers and setting the appropriate flash memory loader, the user can create new code execution or data storage location configurations based on this template.

3.2.2 Optimization levels

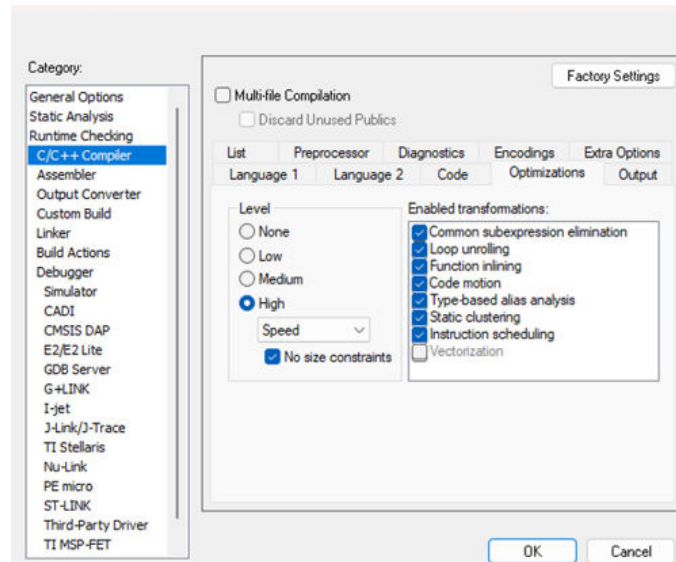
Choosing the right code optimization level can significantly impact the performance and size of the compiled program. Different compilers offer different optimization levels, each with its own trade-offs between compilation time, execution speed, and code size.

Several optimizations level can be chosen and compared:

- None
- Low
- Medium
- High: speed, size, balance

Figure 9 shows the configuration of a project using a "high-by-speed" optimization level with no size constraints.

Figure 9. Optimization level example using EWARM



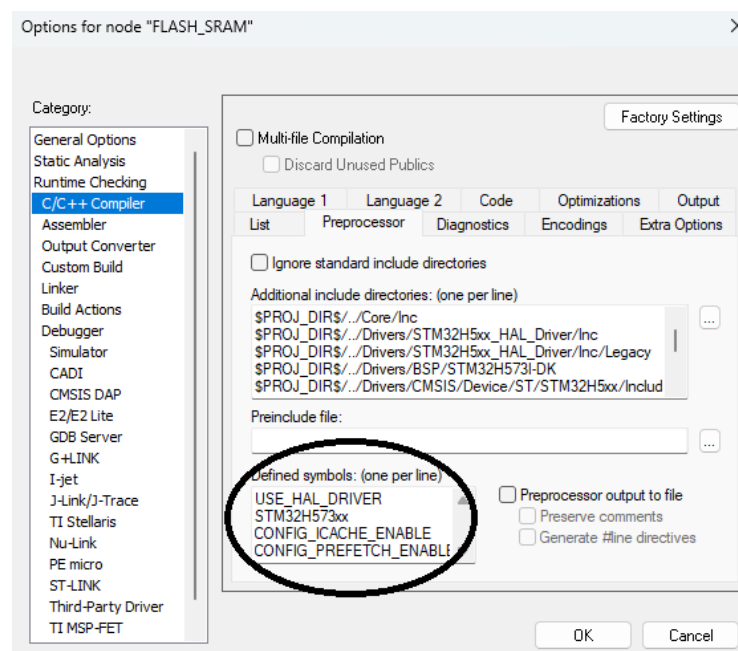
DT75938V1

3.2.3 Preprocessor symbol definition

The use of preprocessing symbols facilitates the configuration of application scenarios by enabling conditional compilation and customization of the code based on specific requirements.

Figure 10 shows the definition of four symbols for STM32H573xx using EWARM. For instance, the "CONFIG_ICACHE_ENABLE" symbol allows the activation or deactivation of cache usage. The same applies for "CONFIG_PREFETCH_ENABLE".

Figure 10. Preprocessor symbol definition example using EWARM



DT75939V1

These preprocessor directives are used in application code to enable conditional compilation. For example, consider the following code snippet:

```
#ifdef CONFIG_ICACHE_ENABLE
/* Enable ICACHE */
  CACHE_Config();
#endif
```

The use of the `#ifdef CONFIG_ICACHE_ENABLE` directive ensures that the code between `#ifdef` and `#endif` is only compiled and included in the final executable if the `CONFIG_ICACHE_ENABLE` macro is defined.

3.3 Cache features

3.3.1 ICACHE

The ICACHE offers two types of caching technique: one-way (direct mapping) and two-way.

By default, the ICACHE is configured in associative operating mode (two-way caching enabled), but it is possible to configure the ICACHE in direct-mapped mode (one-way caching enabled) for applications requiring a very-low-power consumption. The ICACHE configuration is done with the WAYSEL bit in the ICACHE_CR register as follows:

- WAYSEL = 0: direct-mapped operating mode (one-way)
- WAYSEL = 1 (default): associative operating mode (two-way)

Table 6 shows the main characteristics of the ICACHE in the STM32H5 series.

Table 6. STM32H5 series ICACHE characteristics

Parameter	One-way ICACHE	Two-way ICACHE
Cache size (Kbytes)	8	
Cache number of ways	1	2
Cache line width	128 bits	
Number of cache lines	512	256 per way

Cacheable regions and remapping feature

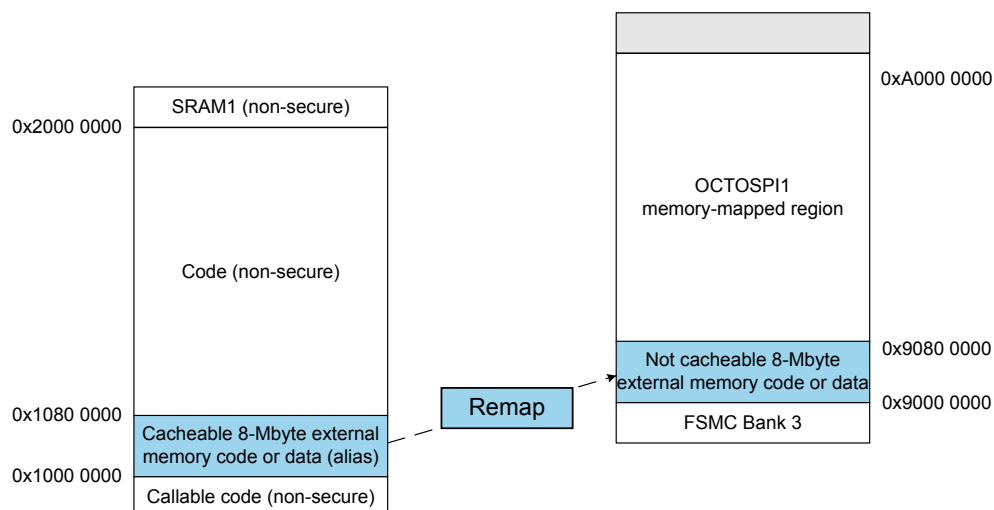
The ICACHE is connected to the Arm® Cortex®-M33 processor through the C-AHB bus and caches the code region of the [0x0000 0000:0x1FFF FFFF] address range.

The external memories are mapped at an address in the range [0x6000 0000:0xAFFF FFFF], so the ICACHE supports a remapping feature. This feature allows any external memory region to be remapped at an address in the range [0x0000 0000:0x07FF FFFF] or [0x1000 0000:0x1FFF FFFF] and to become accessible through the C-AHB bus. It can remap up to four external memory regions. Once a region has been remapped, the remapping operation occurs even if the ICACHE is disabled or if the transaction is not cacheable.

ICACHE external memory remapping

The example in Figure 11 below shows how to benefit from the enhanced performance of the ICACHE during code execution or data read when accessing an external 8-Mbyte external Octo-SPI memory (such as an external flash memory or RAM).

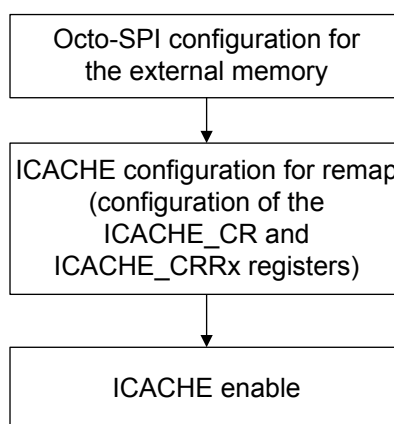
Figure 11. Octo-SPI remapping example



Several steps are necessary to remap an external memory region and make it accessible through the C-AHB bus. Figure 12 below illustrates these steps using Octo-SPI as an example. For more details, refer to the following application notes, available from <http://st.com>:

- *Getting started with Octo-SPI, Hexadeca-SPI, and XSPI Interface on STM32 MCUs* (AN5050)
- *How to use STM32 cache to optimize performance and power efficiency for STM32 MCUs* (AN5212)

Figure 12. Remapping steps



DT75940V1

3.3.2

DCACHE

The purpose of the data cache (DCACHE) is to cache external memory data loads and data stores coming from the processor or from another bus master peripheral. DCACHE manages both read and write transactions.

For STM32H5, the DCACHE slave interface is connected to the Arm® Cortex®-M33 through the S-AHB external memories using FMC and OCTOSPI. The DCACHE is approximately 4 Kbytes.

During boot, DCACHE is disabled by default, so slave memory requests are forwarded directly to the master port. To enable DCACHE, the EN bit must be set in the DCACHE_CR register.

3.4 Configuration requirements of the CPU memory with DMA activation demonstration

The main objective of this task is to perform a transfer involving two masters while overloading the bus. The goal is to test the ability of the system to handle simultaneous data transfers between two master entities and to evaluate the robustness of the bus in high-load conditions. To observe the impact of a multimaster transfer, the FFT is executed using a single DMA transfer (for example, DMA1). This method helps to identify potential weaknesses and improve the overall performance of the system.

To cover many different tests, various parameters and configurations can be defined by following these steps:

1. Configure the different DMA parameters.
2. Check mode: check the configuration and the DMA transfers.
3. Result mode: get the results in term of cycle numbers.

3.4.1 Step 1: configuring the different DMA parameters

There are four configurable parameters for each master:

- Enable/disable of the DMA transfer
- The DMA source address configuration
- The DMA destination address configuration
- The DMA transfer size configuration

Enabling/disabling the DMA transfer

To enable or disable a DMA, the define “#define USE_DMA2” has to be set or cleared to respectively enable or disable the DMA.

DMA source address configuration

The following defines configure the memory source and destination addresses:

- #define DMA_SRC_ADDRESS: the start address of the memory source
- #define DMA_DST_ADDRESS: the start address of the memory destination

The DMA directly transfers data from memory A to memory B.

The DMA_SRC_ADDRESS and DMA_DST_ADDRESS possible values are the following:

- SRAM1_START_ADDRESS
- SRAM2_START_ADDRESS
- SRAM3_START_ADDRESS

DMA destination address configuration

The DMA transfer size must be configured with the following define:

- #define DMA_TRANSFER_SIZE: the transfer unit is byte.

The transfer size must be the smallest value between the available size of the DMA memory source and the available size of the DMA memory destination. A macro min(a,b) can be used for this purpose. The objective is to prevent any DMA access to an invalid memory zone.

DMA transfer size

The DMA_TRANSFER_SIZE possible values are:

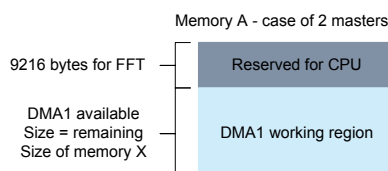
- SRAM1_DMA_AVAILABLE_SIZE
- SRAM2_DMA_AVAILABLE_SIZE
- SRAM3_DMA_AVAILABLE_SIZE

Example of DMA transfer size configuration where the SRAM1 is the source and the SRAM2 is the destination:

- #define DMA_TRANSFER_SIZE min (SRAM1_DMA_AVAILABLE_SIZE, SRAM2_DMA_AVAILABLE_SIZE)

When the user runs DMA[X] (for example, DMA1) with concurrency with the CPU on a given memory (memory A), it must split the memory as shown in Figure 13. The figure shows how to partition memory A (for example the SRAM1) in case of concurrency between two masters in the software. The partitioning shows that memory A is shared between the CPU and the DMA1. This partitioning allows the DMA to complete its transfer after the FFT process to ensure that concurrency is guaranteed during all the FFT process. The RAM size reserved for the FFT is 9216 bytes in the linker file (for CPU access).

Figure 13. How to split memory A in case of concurrency between all the masters



DT71609V2

3.4.2 Step 2: checking the configuration and the DMA transfer(s)

At this stage, all DMA parameters must be set with the necessary values, such as channel attributes, channel transfer mode (normal mode), or interrupt mode (nonblocking mode).

3.4.3 Step 3: getting the results

At this stage, the DMA transfers are done successfully and the user can compile, load, and run it.

The HyperTerminal can be used to display the current configuration, as well as the number of cycles.

4 Results and analysis

4.1 CPU memory access performance

This section explains each feature activation following the configuration used. It also presents the obtained results in terms of the number of cycles consumed by the FFT process for different configurations.

To evaluate the impact of changing code locations on the performance of STM32H5 devices, various configurations are tested. Additionally, to assess the effect of cache usage, the instruction and data caches are enabled or disabled in all tested configurations, using predefined processor symbols.

The tested configurations are:

- Internal memory location:
 - FFT code in flash and read-write data in SRAM1
 - FFT code in SRAM1 (noncacheable addresses) and read-write data in SRAM3
 - FFT code in SRAM1 (cacheable addresses) and read-write data in SRAM3
- External memory location: FFT code and data are in the external OCTO flash memory

For each location, the instruction cache and prefetch are either enabled or disabled using the preprocessor defined symbols.

- Note:*
1. *In order to obtain the maximum of device performance, it is recommended to enable both ICACHE and flash memory prefetch during the use of the flash memory as code execution memory location.*
 2. *The cycle number values may change from one toolchain version to another.*
 3. *These tests are executed with OTFDEC OFF. Enabling this feature may impact the performance.*
 4. *Enabling TrustZone® does not impact the performance of the device.*

4.1.1 Internal memory configuration tests

The following results were obtained in testing with the following configurations:

- Code in flash memory and read-write data in SRAM1
- Code in SRAM1 (noncacheable addresses) and read-write data in SRAM3
- Code in SRAM1 (cacheable addresses) and read-write data in SRAM3

For each location, the instruction cache and prefetch are either enabled or disabled.

All tests respect the following conditions:

- Applicable product: STM32H573 (with an STM32H573I-DK Discovery kit)
- System frequency: 250 MHz
- Compiler: IAR V 9.50.2
- Optimization level: high speed without size constraints

Table 7 shows the results of the FFT execution in terms of the number of cycles for different memory locations.

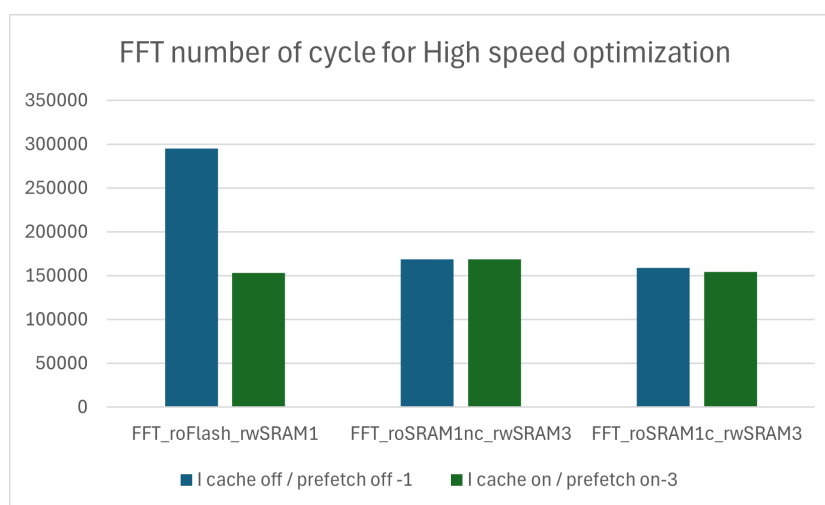
Table 7. Number of FFT cycles for different internal memory locations

FFT Code	FFT Data ⁽¹⁾	ICACHE configuration	Prefetch configuration	Number of cycles
Internal flash memory	Internal SRAMi (where i=1,2 or 3)	Disabled	Disabled	295.142
			Enabled	234.525
	Internal SRAMi (where i=1,2 or 3)	Enabled (2-way)	Disabled	155.724
			Enabled	153.189
SRAM1 Noncacheable addresses (connected through S-bus)	Internal SRAM3	N/A	N/A	168.491
SRAM1 Cacheable addresses (connected through C-bus)	Internal SRAM3	Disabled	N/A	158.934 ⁽²⁾
		Enabled (2-way)		154.454

1. The changing of the location of data between the available SRAMs (1, 2, or 3) does not affect the performance result. The SRAM used for data location is connected through the S-bus.
2. By distributing the FFT data and codes across the two buses, S-bus and C-bus (even with the ICACHE disabled), the number of cycles required for execution is nearly the same as when using flash memory with cache enabled.

Several tests are also conducted to demonstrate the impact of:

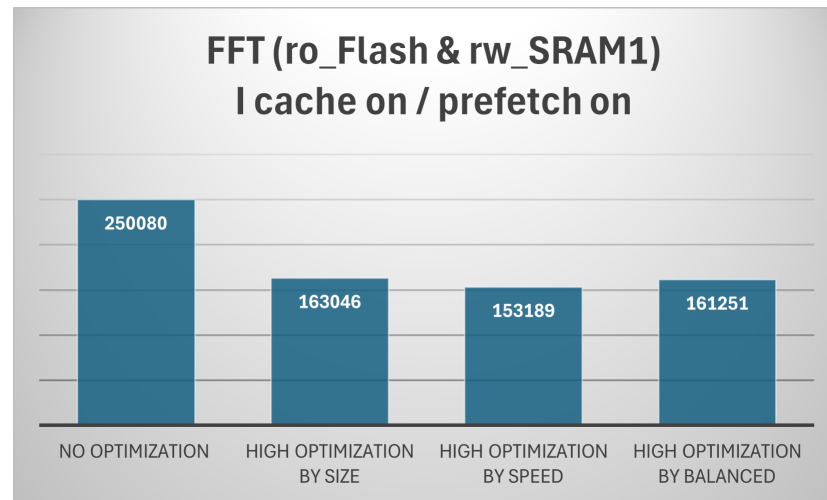
- Using prefetch and ICACHE on the execution of the FFT (as shown in Figure 14).
- Modifying the optimization level (as shown in Figure 15).

Figure 14. ICACHE/prefetch impact on FFT execution time


For the optimization level, the tests are shown for the following configuration:

- Code in flash memory and read-write data in SRAM1
- ICACHE and prefetch enabled

Figure 15. Level optimization impact on FFT execution time



DT75942V1

Analysis

- The best configuration allowing minimum execution time is:
 - Code in flash memory and read-write data in SRAM1.
 - ICACHE and prefetch enabled.
 - Using ICACHE and prefetching techniques results in an approximately 48.09% increase in execution speed (decrease of execution time).
 - Optimizing the compiler for speed can result in a performance improvement of approximately 38.74%.
- Note:*
- The flash memory access time is adjusted to the minimum number of wait states based on the f_{HCLK} frequency (refer to the table 'FLASH recommended number of wait states and programming delay' in the product reference manual).
 - If the code and read-only data are located in one of the SRAMs, the vector table location must also be changed.
 - To use the ICACHE capabilities, the code and data must be mapped to cacheable addresses.
 - Changing the location of data between the available SRAMs (1, 2, or 3) does not affect the performance results.
 - If the application must be executed from the SRAMs, the code must be mapped to a cacheable memory access to ensure fast execution.

4.1.2 External memory configuration tests

The objective of these tests is to measure the performance of the FFT code when it is executed from the external flash memory. To enable execution from external memory, consider the following key aspects:

- Allocate the code and data to the external memory (using the linker file).
- Configure OCTOSPI.
- Specify the external memory characteristics (such as memory frequency and dummy cycles).
- Enable or disable ICACHE/DCACHE use.

For further details, refer to the application note *Getting started with Octo-SPI, Hexadeca-SPI, and XSPI Interface on STM32 MCUs* (AN5050), available from www.st.com.

Test conditions for the execution of the FFT on the external memory:

- FFT code read-write data: external octo-SPI flash memory
- External octo-SPI flash memory: 150 MHz (STR)
- External octo-SPI flash memory: 125 MHz (DTR mode)

Table 8 shows an example configuration where the code is executed from the octo-SPI flash memory using an XSPI2 instance and remapped in the cache addresses.

Table 8. Number of FFT cycles for external memory locations

Transfer mode	FFT code ⁽¹⁾	FFT data ⁽²⁾	ICACHE configuration	DCACHE configuration	Number of cycles
STR	External memory (through noncacheable memory access, connected through S-bus)	Internal SRAM1	Disabled	Disabled	1.550.051
				Enabled	187.730
	External memory (through cacheable memory access, connected through C-bus)	Internal SRAM1	Enabled (2-way)	Disabled	165.649
				Enabled	165.662
DTR	External memory (through noncacheable memory access, connected through S-bus)	Internal SRAM1	Disabled	Disabled	996.166
				Enabled	184.147
	External memory (through cacheable memory access, connected through C-bus)	Internal SRAM1	Enabled (2-way)	Disabled	161.585
				Enabled	161.580

1. Refer to Table 4. *Memory characteristics for the STM32H5 series* for memory access addresses.
2. The used SRAM for data location is connected through the S-bus.

Analysis

- The best configuration is the DTR transfer mode with ICACHE and DCACHE enabled.
- The incorporation of cache features with external memories provides several advantages:
 - Improved data access speed
 - Reduced latency
 - Optimized memory bandwidth
- Using ICACHE can enhance execution time by approximately 83.78%.
- The DTR enhancement compared to STR is significant only when ICACHE is not used, resulting in a 35.73% improvement.

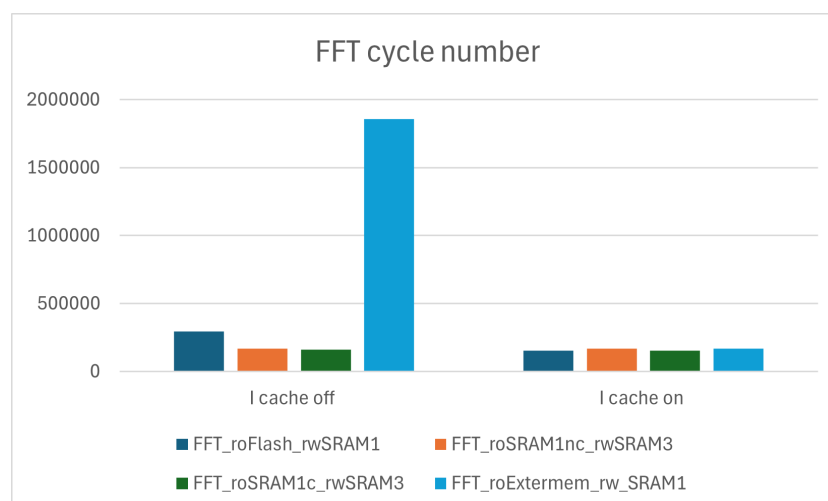
4.1.3 Comparison between internal and external memory locations

This comparison uses the following configuration:

- External octo-SPI flash memory: 125 MHz (DTR mode)

Figure 16 illustrates the impact of ICACHE use for different locations between external and internal memories.

Figure 16. Impact of ICACHE use on FFT cycle count performance



Analysis

Incorporating cache features with external memories results in approximately the same execution time as using the internal memory.

4.2 CPU memory access performance with DMA usage

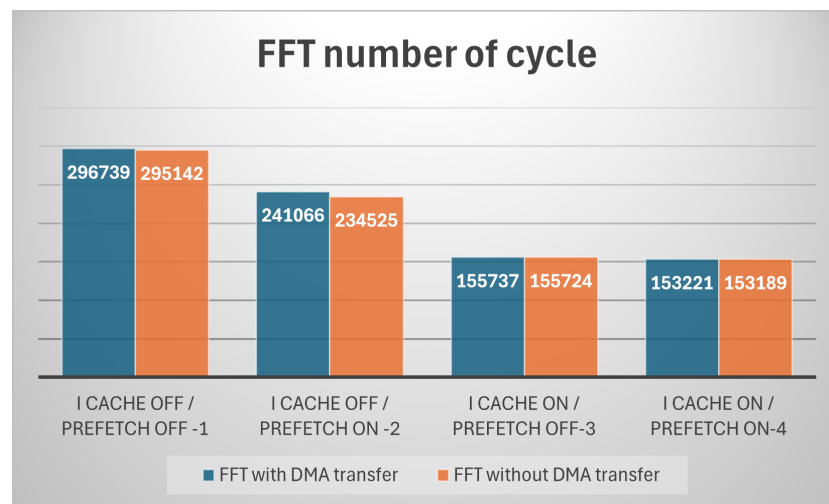
This section treats the performance in the scenario where one or more masters are activated.

Test conditions:

- FFT code: internal flash memory
- FFT data: internal SRAM3
- DMA transfer: from SRAM1 to SRAM3 with 256 ko to be transferred

Figure 17 summarizes the number of cycles consumed by the execution of the FFT process in different scenarios. The results are obtained with five-wait-states access to the internal flash memory.

Figure 17. Impact of ICACHE use on FFT cycle count performance



DT75944V1

Analysis

- If the CPU and one or more masters do not access the same memory, there is no contention.
- When the CPU and one DMA access the same memory, a contention occurs since there is an arbitration either on the bus matrix for memories connected to the bus matrix (SRAM1, SRAM2, etc.), or on the CPU side where the concurrency is performed at the SRAM3 level.
- As the results show, without the use of cache and prefetch, the concurrency of the CPU and DMA1 when writing to or reading from SRAM3 results in a performance degradation of approximately 0.513%.
- By enabling the cache and prefetch, the degradation is reduced from 0.53% to 0.02%.

5 Conclusion

The applications become more complex and require efficient microcontrollers that provide performance. Through its smart and secure architecture, the [STM32H5](#) device is a suitable platform for the internet of things (IoT) applications. It enables the developers in need of a fast microcontroller to ease the code optimization. A better application responsiveness is obtained with the remarkable mechanisms of [STM32H5](#). They enable reaching almost 0-wait states in code execution and a higher performance ranking. Moreover, with the ICACHE/DCACHE capabilities, the user spends less time optimizing the code and the data size by adding external memory resources with no performance penalty. Even if the DMA transfers are done at the same time as the CPU, the [STM32H5](#) system architecture offers a steady performance.

Revision history

Table 9. Document revision history

Date	Version	Changes
29-Jun-2023	1	Initial release.
04-Apr-2024	2	Updated: - Section 2.6.3.2: Octo-SPI interface - Figure 2. STM32H5 series system architecture - Table 1. STM32H5 series cache sizes - Section 2.8: Main differences between the STM32H5 series devices
03-Mar-2025	3	Updated: - Section 2.2: TrustZone® security architecture - Introduction - Table 3. Main differences from architecture point of view - Section 3: Typical application - Section 3.1: FFT demonstration - Section 3.2.1: Memory organization - Section 3.2.3: Preprocessor symbol definition - Table 6. STM32H5 series ICACHE characteristics - Section 3.4: Configuration requirements of the CPU memory with DMA activation demonstration - Section 3.4.1: Step 1: configuring the different DMA parameters - Section 4.1: CPU memory access performance - Section 4.2: CPU memory access performance with DMA usage - Section 5: Conclusion Added: - Section 3.2: Configuration requirements of the CPU access demonstration - Section 3.3: Cache features - Section 4.1.1: Internal memory configuration tests - Section 4.1.2: External memory configuration tests - Section 4.1.3: Comparison between internal and external memory locations

Contents

1	General information	2
2	STM32H5 series system architecture	3
2.1	Cortex [®] -M33 core	3
2.2	TrustZone [®] security architecture	3
2.3	Instruction and data caches	4
2.3.1	Instruction cache	4
2.3.2	Data cache	4
2.4	Cortex [®] -M33 bus interfaces	4
2.4.1	Code AHB (C-AHB) interface	5
2.4.2	System AHB (S-AHB) interface	5
2.5	STM32H5 bus matrix	7
2.6	STM32H5 memories	8
2.6.1	Embedded flash memory	8
2.6.2	Embedded SRAM	9
2.6.3	External memories	10
2.7	DMAs	12
2.8	Main differences between the STM32H5 series devices	13
3	Typical application	14
3.1	FFT demonstration	14
3.2	Configuration requirements of the CPU access demonstration	14
3.2.1	Memory organization	14
3.2.2	Optimization levels	16
3.2.3	Preprocessor symbol definition	17
3.3	Cache features	18
3.3.1	ICACHE	18
3.3.2	DCACHE	19
3.4	Configuration requirements of the CPU memory with DMA activation demonstration	20
3.4.1	Step 1: configuring the different DMA parameters	20
3.4.2	Step 2: checking the configuration and the DMA transfer(s)	21
3.4.3	Step 3: getting the results	21
4	Results and analysis	22
4.1	CPU memory access performance	22
4.1.1	Internal memory configuration tests	22
4.1.2	External memory configuration tests	24
4.1.3	Comparison between internal and external memory locations	25

4.2	CPU memory access performance with DMA usage	26
5	Conclusion	27
	Revision history	28
	List of tables	31
	List of figures.....	32

List of tables

Table 1.	STM32H5 series cache sizes	4
Table 2.	Memory access behavior	6
Table 3.	Main differences from architecture point of view	13
Table 4.	Memory characteristics for the STM32H5 series	15
Table 5.	Linker file extensions	15
Table 6.	STM32H5 series ICACHE characteristics	18
Table 7.	Number of FFT cycles for different internal memory locations	23
Table 8.	Number of FFT cycles for external memory locations	25
Table 9.	Document revision history	28

List of figures

Figure 1.	Armv8-M security extension	3
Figure 2.	STM32H5 series system architecture	7
Figure 3.	Flash memory interfaces (paths: 1, 2, 3)	9
Figure 4.	SRAMs possible connections (paths: 3, 4 and 5).	10
Figure 5.	External memory connections.	10
Figure 6.	External memory based on IDAU mapping	11
Figure 7.	FFT example block diagram	14
Figure 8.	ICF file mapping memory regions	16
Figure 9.	Optimization level example using EWARM	17
Figure 10.	Preprocessor symbol definition example using EWARM.	17
Figure 11.	Octo-SPI remapping example.	19
Figure 12.	Remapping steps	19
Figure 13.	How to split memory A in case of concurrency between all the masters	21
Figure 14.	ICACHE/prefetch impact on FFT execution time	23
Figure 15.	Level optimization impact on FFT execution time	24
Figure 16.	Impact of ICACHE use on FFT cycle count performance	25
Figure 17.	Impact of ICACHE use on FFT cycle count performance	26

IMPORTANT NOTICE – READ CAREFULLY

STMicroelectronics NV and its subsidiaries ("ST") reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice. Purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST's terms and conditions of sale in place at the time of order acknowledgment.

Purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of purchasers' products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2025 STMicroelectronics – All rights reserved